

Elevate Labs Project : Web Application Vulnerability Scanner

Sahil Borse

28 July 2025

1 Introduction

Developed as part of the Elevate Labs Cybersecurity Internship (July 2025), this project delivers a Python-based web application vulnerability scanner to identify common security flaws, including Cross-Site Scripting (XSS), SQL Injection (SQLi), and Cross-Site Request Forgery (CSRF). Aligned with the OWASP Top 10, the scanner uses a web interface to facilitate vulnerability detection and reporting, enhancing web application security testing.

2 Abstract

The scanner crawls target websites to identify input fields and URLs, injects payloads to detect XSS and SQLi vulnerabilities, and checks for CSRF token absence. Built with Python, it leverages `requests` for HTTP interactions, `BeautifulSoup` for HTML parsing, and `Flask` for a user-friendly web interface. Results are displayed in a table, logged with evidence and severity, and exportable for further analysis, providing actionable insights for securing web applications.

3 Tools Used

- **Python:** Core language for scripting, using `requests` for HTTP requests, `BeautifulSoup` for HTML parsing, and `Flask` for the web interface.
- **OWASP Top 10:** Guided detection of XSS (A7), SQLi (A1), and CSRF (A8).
- **Ubuntu 22.04:** Development environment hosted in a VirtualBox VM.
- **LaTeX:** Used for generating the project report in PDF format.

4 Steps Involved

1. **Environment Setup:** Created a project directory (`vuln_scanner`) in `/Elevate-Labs-Final-Projects` and set up a Python virtual environment with `requests`, `beautifulsoup4`, and `flask`.
2. **Crawler Development:** Implemented `crawler.py` to extract forms and URLs from a target website using `BeautifulSoup`, limiting crawl depth to prevent excessive requests.
3. **Vulnerability Scanning:** Developed `scanner.py` to inject XSS and SQLi payloads, using regex for detection, and check for CSRF token absence. Fixed a regex syntax error in the XSS detection function to ensure accurate pattern matching.
4. **Flask UI:** Built a `Flask` web interface (`app.py`, `scan.html`) running on port 5001, allowing users to input URLs, view scan results, and download a vulnerability log.
5. **Logging and Reporting:** Configured logging to `vulnerabilities.log` with timestamps, vulnerability types, payloads, severity, and evidence. Generated a PDF report using LaTeX.

5 Conclusion

The Web Application Vulnerability Scanner effectively identifies XSS, SQLi, and CSRF vulnerabilities, providing a user-friendly interface for security testing. The project demonstrates practical application of OWASP principles and Python-based automation. Future improvements could include support for

additional vulnerabilities (e.g., LFI, RFI) and integration with automated remediation tools. The scanner is a valuable tool for assessing web application security in controlled environments.